AI FITNESS COACH: AN INTELLIGENT EVIDENCE-GROUNDED EXERCISE KINEMATICS AND NUTRITION COACHING PLATFORM

COMPREHENSIVE TECHNICAL PROJECT REPORT AND SYSTEM SPECIFICATION

Academic Year: 2025–2026
Document Classification: Final Project Technical Documentation & Faculty Evaluation Report
Target Systems: Web-Based Client (React 18 + Vite), Machine Learning Edge Runtime (MediaPipe Tasks Vision Wasm/GPU), Real-Time Relational Backend (FastAPI + SQLAlchemy + SQLite), Vector Search Engine (FAISS + SentenceTransformers), and Retrieval-Augmented Generation Service (Google GenAI Gemini).
Repository Location: `ai-fitness-coach`

FACULTY EVALUATION AND APPROVAL

Evaluation Metric	Maximum Score	Awarded Score	Faculty Remarks
System Architecture & Design	20		
Computer Vision & Kinematics Implementation	25		
AI RAG Pipeline & Biometric Evidence Grounding	20		
Backend APIs & Database Integrity	15		
Code Quality, Testing & Verification	10		
Documentation & Technical Presentation	10		
TOTAL SCORE	**100**		

Evaluator Name: _____
Signature & Date: _____

TABLE OF CONTENTS

1. Introduction and Project Objectives
   - 1.1 Problem Statement
   - 1.2 Project Objectives
   - 1.3 Target Audience and Use Cases
   - 1.4 Scope and Boundaries
2. Technology Stack and System Architecture
   - 2.1 Technology Stack Inventory
   - 2.2 High-Level System Architecture
   - 2.3 Component Interaction and Data Flow Diagram
   - 2.4 Edge Computer Vision Pipeline Architecture
   - 2.5 Evidence-Grounded RAG Subsystem Architecture
3. Installation, Configuration, and Application Startup
   - 3.1 Hardware and Environment Prerequisites
   - 3.2 Backend Setup and Dependency Installation
   - 3.3 Evidence Vector Corpus Ingestion

- 3.4 Frontend Setup and Configuration
- 3.5 Launching Services and Port Verification

4. Complete Feature Documentation
   - 4.1 Secure Authentication and Role-Based Access Control
   - 4.2 Biometric Onboarding and Athlete Health Profiling
   - 4.3 Interactive Main Athlete Dashboard
   - 4.4 Live Workout Tracking and Computer Vision Feedback
   - 4.5 Biomechanical Kinematic Analyzers (All Six Exercises)
   - 4.6 Multi-Stage State Machine and Anti-Bounce Repetition Filtering
   - 4.7 Evidence-Grounded AI Fitness Chatbot (RAG System)
   - 4.8 Scientific Nutrition, Water, and Diet Management
   - 4.9 Workout History, Progression Analytics, and Benchmarks
   - 4.10 Platform Administration, Diagnostics, and Audit Logs
5. Step-by-Step Workflows and Procedural Specifications
   - 5.1 User Registration and Authentication Workflow
   - 5.2 User Onboarding and Health Profile Generation
   - 5.3 Live Workout Pose Detection and Real-Time Tracking Workflow
   - 5.4 Barbell Deep Squat Kinematic Analysis Workflow
   - 5.5 Chest Push-Up Kinematic Analysis Workflow
   - 5.6 Romanian Deadlift Kinematic Analysis Workflow
   - 5.7 Dumbbell Bicep Curl Kinematic Analysis Workflow
   - 5.8 Forward Walking Lunge Kinematic Analysis Workflow
   - 5.9 Isometric Core Plank Kinematic Analysis Workflow
   - 5.10 Evidence-Grounded AI Chatbot Query and RAG Processing
   - 5.11 Nutrition Logging, Target Adjustment, and Hydration Tracking
   - 5.12 Platform Administration, User Moderation, and Diagnostics
6. Backend APIs, Database Models, and Project File Structure
   - 6.1 Database Architecture and Entity Relationship Model
   - 6.2 Complete RESTful API Endpoint Catalog
   - 6.3 Project File Inventory and Architectural Organization
7. Testing, Verification, Limitations, and Troubleshooting
   - 7.1 Automated Unit and Biomechanical Test Suite
   - 7.2 Backend Database and Service Verification Tests
   - 7.3 Live Chatbot Latency and RAG Benchmark Verification
   - 7.4 Production Build and Static Lint Verification
   - 7.5 Implemented vs. Unverified/Incomplete Features
   - 7.6 Known Limitations and Biomechanical Constraints
   - 7.7 System Troubleshooting Guide
8. Conclusion and Future Roadmap
   - 8.1 Summary of Contributions
   - 8.2 Future Enhancements

---

1. INTRODUCTION AND PROJECT OBJECTIVES

1.1 Problem Statement

Physical exercise is essential for cardiovascular health, musculoskeletal strength, and metabolic disease prevention. However, unsupervised resistance training carries substantial risks:

1. **Injuries from Improper Biomechanical Form:** Lifting weights or performing bodyweight movements with incorrect joint alignment (e.g., knee valgus during squats, lumbar hyper-extension during push-ups, or rounding the thoracic spine during deadlifts) frequently leads to acute ligament tears, disc herniations, and chronic joint degeneration.
2. **High Cost and Inaccessibility of Personal Trainers:** Certified human trainers provide real-time form correction, but personal coaching is economically prohibitive for the majority of individuals, costing $50 to $120 per session.
3. **Pervasive Fitness Disinformation and Generative AI Hallucinations:** When gym-goers turn to generic conversational AI chatbots, these models routinely hallucinate unverified fitness advice, invent non-existent scientific studies, or suggest dangerous dietary deficits and excessive loads that ignore user injuries.
4. **Fragmented Fitness Ecosystems:** Modern fitness enthusiasts are forced to juggle multiple disjointed applications: one for rep counting, one for diet logging, and another for fitness articles, resulting in poor adherence and data fragmentation.

1.2 Project Objectives

The **AI Fitness Coach** web platform resolves these limitations through an integrated, multi-disciplinary system:

- **Client-Side Edge Computer Vision:** Execute monocular pose estimation entirely in the client browser using WebAssembly (Wasm) and WebGL GPU acceleration via Google MediaPipe Tasks Vision, eliminating video streaming latency and safeguarding user privacy.
- **Form-Validated Kinematic Repetition Counting:** Implement mathematical joint angle calculation and multi-frame state machines across six foundational exercises (Squats, Push-ups, Deadlifts, Bicep Curls, Lunges, and Planks) with strict parallel depth verification, anti-bounce tempo validation, and audio-visual coaching cues.
- **Evidence-Grounded RAG AI Coaching:** Build a Retrieval-Augmented Generation (RAG) pipeline powered by Facebook AI Similarity Search (FAISS) and SentenceTransformers (`all-MiniLM-L6-v2`) over 123 peer-reviewed sports science chunks. Query Google Gemini with strict negative constraints, ensuring advice cites recognized authorities (ACSM, BJSM, ISSN, WHO) with verified DOIs/PMIDs and never fabricates claims.
- **Holistic Training, Nutrition, and Analytics Tracking:** Provide full-stack relational persistence via FastAPI and SQLite for workout history, macronutrient targets, water consumption, and personal benchmarks.
- **Administrative Governance:** Implement a dedicated operations portal for user status management, hardware health diagnostics, and platform audit logging.

1.3 Target Audience and Use Cases

- **Home Gym and Remote Athletes:** Individuals training without a live coach who need real-time auditory cues on depth, posture, and rep counts.
- **Beginner Exercisers:** Novices learning foundational compound movement mechanics safely with immediate corrective prompts before loading heavy weights.
- **Sports Science Students and Researchers:** Athletes seeking scientifically validated recommendations on progressive overload, protein dosing, and hydration strategies with formal peer-reviewed citations.
- **Gym and Enterprise Administrators:** Fitness club staff monitoring athlete adherence, tracking platform utilization, and auditing system throughput.

1.4 Scope and Boundaries

- **In Scope:** Monocular 2D video landmark extraction via standard webcams; deterministic angle calculations on sagittal and frontal planes; 6 exercise analyzers; local FAISS vector search; Gemini LLM generation with citation enforcement; SQLite persistence; responsive dark-mode UI.
- **Out of Scope:** Multi-camera 3D motion-capture studio triangulation; medical diagnostic prescription for acute injuries; proprietary wearable hardware integration (e.g., smartwatches, EMG electrodes).

---

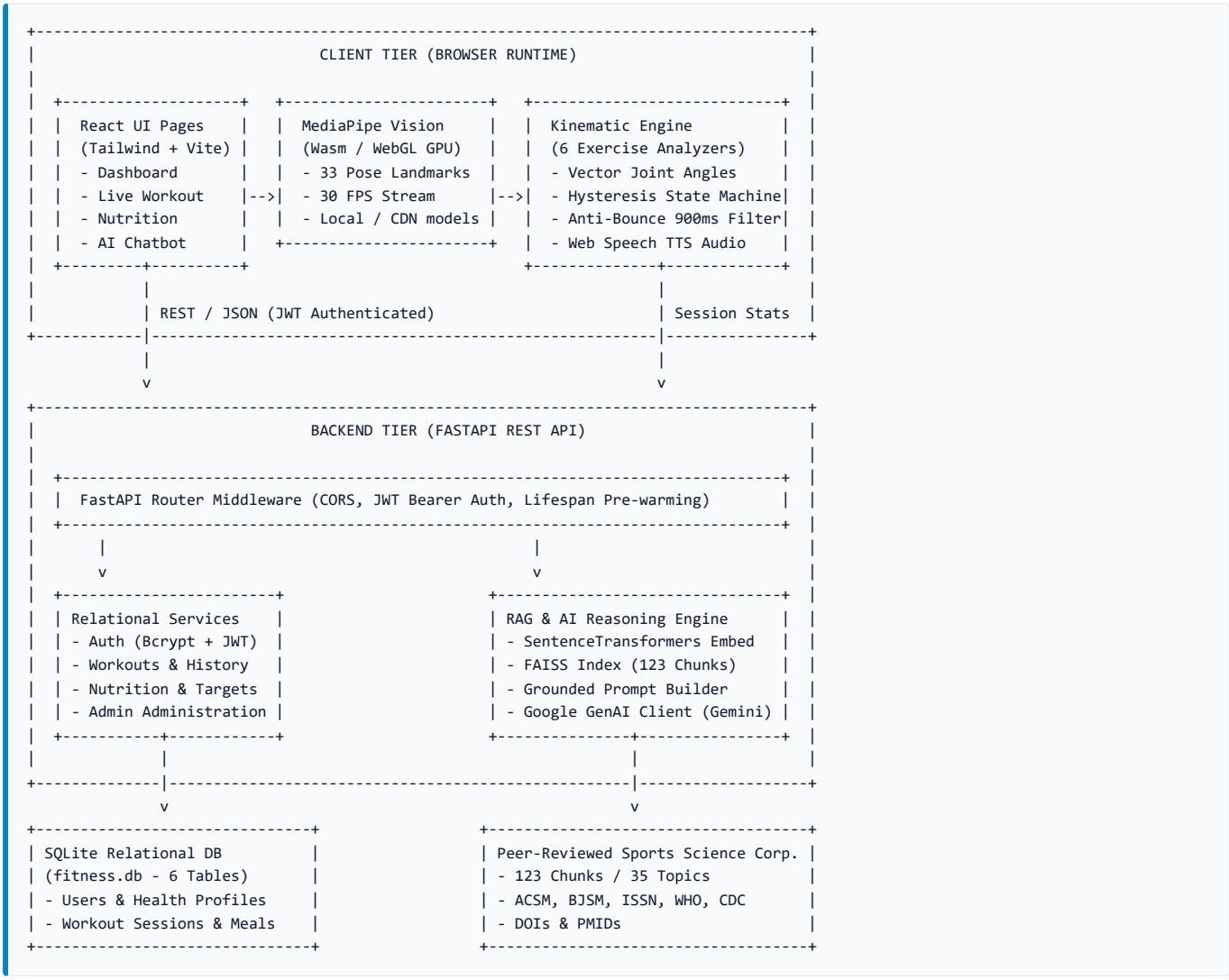
2. TECHNOLOGY STACK AND SYSTEM ARCHITECTURE

2.1 Technology Stack Inventory

Architectural Tier	Technology / Library	Version	Purpose in Application
Frontend Framework	React.js	`18.3.1`	Component-based reactive UI rendering and state management
Build Tool & Bundler	Vite	`5.4.2`	High-performance Hot Module Replacement (HMR) and production bundling
Styling & Design System	Tailwind CSS	`3.4.1`	Bespoke high-contrast dark theme, typography, and responsive layouts
Icons & Visual Assets	Lucide React	`0.344.0`	Comprehensive iconography for UI controls and biometric status indicators
Data Visualization	Chart.js & React-Chartjs-2	`4.4.1` / `5.2.0`	Interactive charts for form trends, rep distributions, and weight metrics
Client-Side Vision ML	`@mediapipe/tasks-vision`	`0.10.14`	Client-side 33-point human skeletal pose landmark extraction
Celebration Effects	Canvas Confetti	`1.9.4`	Gamified visual feedback upon workout completion
Backend REST Engine	FastAPI	`0.141.1`	High-throughput asynchronous REST API and dependency injection
ASGI Web Server	Uvicorn	`0.52.4`	Production-grade ASGI server handling asynchronous HTTP requests
Data Modeling & ORM	SQLAlchemy	`2.1.6`	Object-Relational Mapping for SQLite with cascade relationships
Data Validation	Pydantic	`2.13.5`	Strict schema parsing, type coercion, and request validation
Security & Tokens	Bcrypt & PyJWT	`5.0.0` / `2.15.1`	Salting (12 rounds) and HMAC-SHA256 JWT access tokens
Vector Search Engine	FAISS CPU (`faiss-cpu`)	`1.15.0`	Fast inner-product cosine similarity retrieval across vector embeddings
Dense Embeddings	SentenceTransformers	`6.0.1`	`all-MiniLM-L6-v2` 384-dimensional dense semantic vector encoding
Generative LLM SDK	Google GenAI SDK	`2.22.0`	Multi-model fallback reasoning (`gemini-3.5-flash-lite` , `gemini-3.5-flash`)

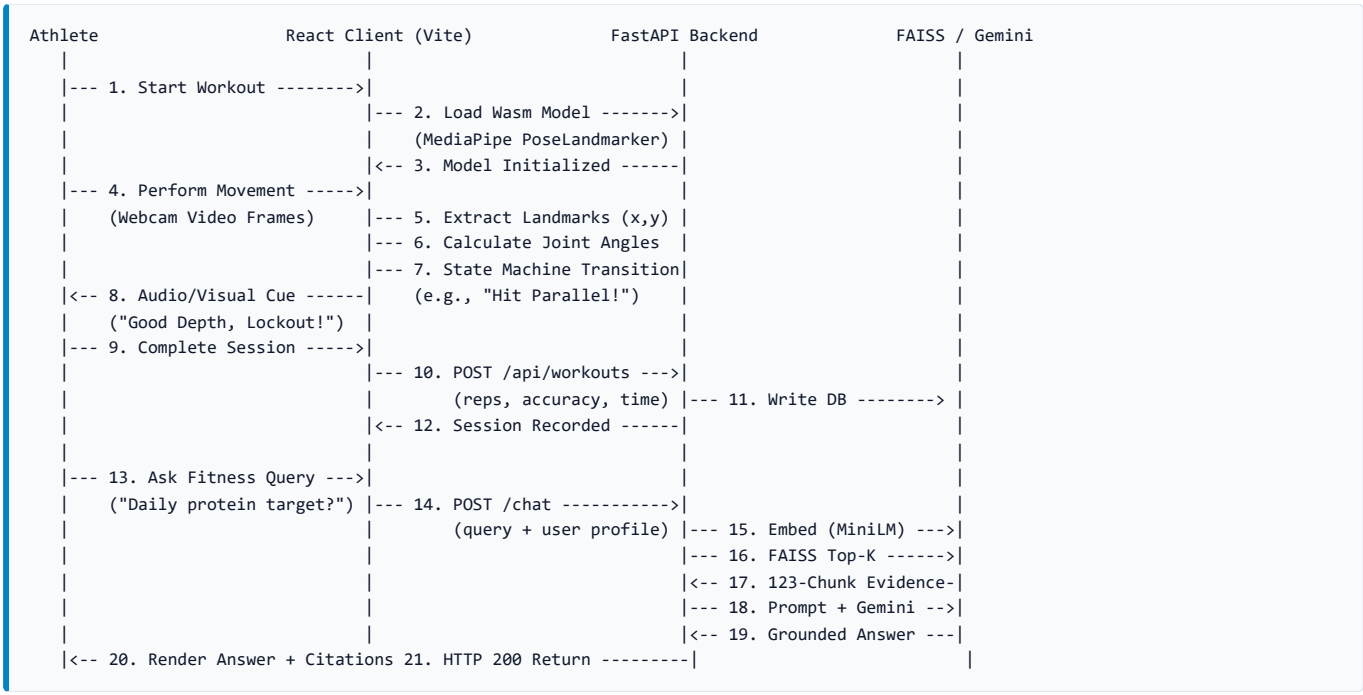
2.2 High-Level System Architecture

The application adheres to a decoupled client-server architecture with client-side edge computing for latency-critical computer vision tasks and server-side processing for dense vector retrieval, LLM synthesis, and relational persistence.



2.3 Component Interaction and Data Flow Diagram

The sequence below illustrates how a user request traverses the system during a Live Workout and an AI Chatbot interaction:



2.4 Edge Computer Vision Pipeline Architecture

Running computer vision at the edge in the browser guarantees real-time responsiveness (<33ms frame times for 30 FPS tracking) and complete privacy:

1. **Video Capture:** HTML5 `navigator.mediaDevices.getUserMedia({ video: { width: 1280, height: 720 } })` pipes camera frames to a hidden HTML `<video>` element.
2. **Local Model Resolution:** The `getPoseLandmarker()` singleton initializes `@mediapipe/tasks-vision` by loading local Wasm binaries (`/wasm`) and model weights (`/models/pose_landmarker_lite.task`). If local files are inaccessible, it transparently falls back to Google's CDN.
3. **Hardware Acceleration:** The landmarker requests a WebGL `GPU` delegate. If unavailable, it degrades gracefully to multi-threaded CPU SIMD.
4. **Landmark Extraction:** Every frame yields 33 3D normalized coordinates (x, y, z) with detection and visibility confidence metrics.
5. **Canvas Overlay Rendering:** Coordinates are drawn to an overlaid HTML5 `<canvas>` via `PoseCanvas.jsx` , highlighting skeletal segments with dynamic color states (green for compliant joints, amber for warning thresholds, red for form errors).

2.5 Evidence-Grounded RAG Subsystem Architecture

To eliminate LLM hallucinations:

1. **Curated Scientific Corpus:** 123 evidence passages extracted from peer-reviewed position stands (ACSM, BJSM, ISSN, WHO) covering 35 fitness and nutrition topics.
2. **Vector Space Indexing:** Texts are encoded via `all-MiniLM-L6-v2` into 384-dimensional normalized vectors and indexed via FAISS `IndexFlatIP` (inner product cosine similarity).
3. **Retrieval Gating:** User queries are vectorized at runtime (~17ms) and queried against FAISS (~0.2ms). Chunks scoring below a strict similarity threshold ($\text{score} < 0.40$) are pruned.
4. **Boundary-Constrained Prompting:** The LLM prompt combines retrieved evidence, user biometrics, and 16 strict negative rules:
   - Must not invent statistics, studies, or quantitative numbers not explicitly present in the text.
   - Must explicitly refuse medical diagnosis or pain management prescriptions.
   - Must output formal citations including Author, Journal, Year, PMID, and DOI.
5. **Multi-Model Failover:** The backend targets `gemini-3.5-flash-lite` (average latency 1.1s–2.8s), with automated instant failover to `gemini-3.5-flash` and `gemini-3.6-flash` .

---

3. INSTALLATION, CONFIGURATION, AND APPLICATION STARTUP

3.1 Hardware and Environment Prerequisites

- **Operating System:** Windows 10/11, macOS 12+, or Ubuntu Linux 22.04 LTS.
- **Processor (CPU):** Quad-Core x86_64 or Apple Silicon (M1/M2/M3).
- **RAM:** Minimum 8 GB (16 GB recommended for running SentenceTransformers and Vite concurrently).
- **Camera Hardware:** Standard integrated or external USB 720p/1080p webcam (or built-in simulation mode).
- **Software Runtimes:**
  - Node.js: `v18.0.0` or higher (verified with Node `v20.x` / `v22.x`)
  - Python: `3.10.x` to `3.13.x`
  - Package Managers: `npm` (Node) and `pip` (Python)

3.2 Backend Setup and Dependency Installation

```
# 1. Navigate to the backend directory
cd c:\Users\ajitk\.gemini\antigravity-ide\scratch\ai-fitness-coach\backend

# 2. Create and activate a Python virtual environment
python -m venv venv
.\venv\Scripts\activate

# 3. Install required Python packages
pip install fastapi uvicorn sqlalchemy pydantic bcrypt pyjwt python-dotenv
pip install faiss-cpu sentence-transformers torch google-genai tenacity requests
```

Create or verify the `backend/.env` file:

```
GEMINI_API_KEY=your_actual_google_gemini_api_key_here
JWT_SECRET=pulseai_super_secret_production_ready_jwt_signing_key_32bytes_long
```

3.3 Evidence Vector Corpus Ingestion

Before running the server, build the FAISS vector index from the scientific evidence corpus:

```
# Run ingest script from backend directory
python ingest.py
```

Expected Output:

```
Indexed 123 evidence chunks.
FAISS index: .../backend/index/fitness.index
Metadata:    .../backend/index/documents.json
```

3.4 Frontend Setup and Configuration

```
# Navigate to the project root directory
cd c:\Users\ajitk\.gemini\antigravity-ide\scratch\ai-fitness-coach

# Install Node dependencies
npm install
```

3.5 Launching Services and Port Verification

Step 1: Launch FastAPI Backend Server

```
cd backend
.\venv\Scripts\activate
uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

Verification Check: Access `http://127.0.0.1:8000/` in your browser. It should return:

```
{"message": "AI Fitness Coach API is running", "database": "active"}
```

Step 2: Launch Vite Frontend Dev Server

```
# In a separate terminal at project root:
npm run dev
```

Verification Check: Access `http://localhost:5173/` or `http://localhost:5174/` to view the landing page.

---

4. COMPLETE FEATURE DOCUMENTATION

4.1 Secure Authentication and Role-Based Access Control

The application implements dual-role security (Athletes and Platform Administrators).

- **Password Hashing:** Passwords are never stored in plaintext. They are salted and hashed using Bcrypt with 12 calculation rounds (`bcrypt.gensalt(rounds=12)`).
- **JWT Token Issuance:** Upon successful credential validation, the server generates an HMAC-SHA256 JSON Web Token containing user ID, email, role, and a 7-day expiration timestamp (`ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 * 7`).
- **Role Verification:** Protected endpoints verify tokens using FastAPI's dependency injection (`get_current_user` and `require_admin`). Unauthorized requests trigger `HTTP 401 Unauthorized` or `HTTP 403 Forbidden` . Suspended accounts are immediately denied access.

```
<div class="screenshot-container">
  <div class="screenshot-placeholder">
    <div class="camera-icon">📷</div>
    <div class="placeholder-title">[SCREENSHOT 1: Login and Registration Page]</div>
    <div class="placeholder-subtitle">High-Resolution Web Interface Capture Area (16:9 Aspect Ratio)</div>
  </div>
  <div class="screenshot-caption">Figure 4.1 - Dual-Role Authentication and Registration Interface.</div>
  <div class="screenshot-note"><strong>Evaluation Note:</strong> This screenshot should display the sleek dark-themed login interface fe
```</div>

4.2 Biometric Onboarding and Athlete Health Profiling
New athletes undergo a structured four-stage biometric profiling workflow that tailors their calorie calculations, training schedules, a
1. **Biometrics & Physical Stats:** Age, Gender, Height (cm), Weight (kg).
2. **Fitness Ambitions:** Goals (Muscle Hypertrophy, Strength & Power, Fat Loss & Conditioning, Posture Correction & Mobility) and Fitne
3. **Logistics & Equipment:** Weekly Frequency (2 to 6 days/week) and Available Equipment (Bodyweight Only, Dumbbells & Bands, Full Comm
4. **Injury & Physical Limitations:** Chronic back, knee, shoulder, or neck pain records.

Data is stored in the `health_profiles` SQLite table and injected into the system prompt during AI chat interactions to personalize reco

4.3 Interactive Main Athlete Dashboard
The athlete dashboard acts as the daily mission control:
* **Metric Cards:** Visualizes current workout streak (days), total sessions completed, total reps accumulated, and average form accurac
* **Today's Workout Preview:** Displays scheduled exercises with set and rep targets, target muscle groups, and a direct "Launch Live Wo
* **Quick Nutrition Progress:** Displays progress rings for daily calories, protein, carbohydrates, fat, and a quick-log meal button.
* **AI Coach Recommendation:** Dynamic coaching advice reflecting the user's selected primary goal.



[SCREENSHOT 2: Main Dashboard]

HIGH-RESOLUTION WEB INTERFACE CAPTURE AREA (16:9 ASPECT RATIO)

Figure 4.2 - Athlete Mission Control Dashboard.

Evaluation Note: *This screenshot should capture the complete athlete dashboard, including streak counters, average form accuracy metrics, today's workout launch card, real-time nutrition macro rings, and active weekly schedule overview.* ```

4.4 Live Workout Tracking and Computer Vision Feedback

The Live Workout interface transforms standard webcams into an interactive smart fitness mirror:

- **Pose Estimation Stream:** Overlays a responsive HTML5 canvas on top of the live video stream, rendering the 33 MediaPipe pose landmarks and connecting skeletal vectors in real time.
- **Dual Operation Modes:**
  - *Webcam Mode:* Reads real-time video frames from the user's physical camera.
  - *Simulation Mode:* Provides a built-in mathematical kinematic landmark generator that cycles through lifelike rep trajectories, enabling automated testing and demonstration without camera hardware.
- **FormFeedbackHUD:** Displays real-time angle readouts, an arc gauge depicting proximity to target depth, repetition counters, set counters, and color-coded status badges (Green = Optimal, Amber = Warning, Red = Posture Error).
- **Text-to-Speech (TTS) Auditory Coach:** Leverages the Web Speech Synthesis API (`window.speechSynthesis`) to provide spoken audio feedback during exercise execution (e.g., "*Good depth, drive up!*", "*Chest falling forward, stand tall!*"). Spoken cues are debounced with a 2.8-second cooldown to avoid sensory overload.
- **Session Completion Celebration:** Triggers canvas confetti upon completion of target sets and displays a detailed performance summary modal.

```
<div class="screenshot-container">
  <div class="screenshot-placeholder">
    <div class="camera-icon">📷</div>
    <div class="placeholder-title">[SCREENSHOT 4: Live Workout and Pose Detection]</div>
    <div class="placeholder-subtitle">High-Resolution Web Interface Capture Area (16:9 Aspect Ratio)</div>
  </div>
  <div class="screenshot-caption">Figure 4.3 - Live Workout Screen with MediaPipe Pose Estimation Overlay.</div>
  <div class="screenshot-note"><strong>Evaluation Note:</strong> This screenshot should illustrate the live camera view with the overla
```` ```</div>
</div>

4.5 Biomechanical Kinematic Analyzers (All Six Exercises)
The system features six specialized analyzer classes in `src/services/squatAnalyzer.js` managed by a unified `ExerciseAnalyzer` facade:
```



**[SCREENSHOT 5: Exercise Analysis and Rep Counter]**

HIGH-RESOLUTION WEB INTERFACE CAPTURE AREA (16:9 ASPECT RATIO)

**Figure 4.4 - Real-Time Biomechanical Rep Counter and HUD Feedback.**

***Evaluation Note:*** *This screenshot should capture the FormFeedbackHUD during an active repetition, showing the real-time angle readout gauge, current rep count, set progress, and dynamic form correction cues.* ```

1. Barbell Deep Squat ( `SquatAnalyzer` )

- **Monitored Joints:** Hip Flexion ($\angle \text{Shoulder-Hip-Knee}$) and Knee Flexion ($\angle \text{Hip-Knee-Ankle}$).
- **Biomechanical Criteria:**
  - Standing Lockout: Knee angle $\ge 148^\circ$.
  - Descent Initiation: Knee angle drops $< 130^\circ$.
  - Target Parallel Depth: Knee angle must reach $\le 100^\circ$ (hip crease below patella).
  - Ascent Phase: Knee angle opens back past $> 115^\circ$.
  - Full Rep Completion: Returns to lockout $\ge 148^\circ$.
- **Form Error Detection:** Flags excessive forward torso lean if hip angle drops below $50^\circ$ before parallel depth is achieved.

2. Chest Push-Up ( `PushupAnalyzer` )

- **Monitored Joints:** Bilateral Elbow Flexion ($\angle \text{Shoulder-Elbow-Wrist}$) and Spine Alignment.
- **Biomechanical Criteria:**
  - Top Lockout: Elbow angle $\ge 150^\circ$ in straight-arm plank.
  - Descent Phase: Elbow angle drops $< 130^\circ$.
  - Bottom Depth: Elbow angle must reach $\le 90^\circ$ (chest approximately 2 inches above the floor).
  - Lockout Return: Elbow returns to $\ge 150^\circ$.
- **Form Error Detection:** Flags incomplete chest descent and flared elbows.

3. Romanian Deadlift ( `DeadliftAnalyzer` )

- **Monitored Joints:** Hip Hinge Angle ($\angle \text{Shoulder-Hip-Knee}$) and Knee Soft-Bend ($\angle \text{Hip-Knee-Ankle}$).
- **Biomechanical Criteria:**
  - Upright Stand: Hip angle $\ge 150^\circ$.
  - Hinge Initiation: Hip angle drops $< 135^\circ$ while maintaining knee angle $> 115^\circ$.
  - Posterior Chain Target Depth: Hip angle reaches $\le 75^\circ$ with bar near mid-shin.
  - Standing Lockout: Hips drive forward to $\ge 150^\circ$.
```

- **Form Error Detection:** If the knee angle drops below 115° during hinging, the system triggers the corrective cue: *"Hinge at your hips, don't squat the deadlift!"*.

4. Dumbbell Bicep Curl (`CurlAnalyzer`)

- **Monitored Joints:** Elbow Flexion ($\angle \text{Shoulder-Elbow-Wrist}$).
- **Biomechanical Criteria:**
  - Full Extension: Elbow angle $\ge 140^\circ$.
  - Flexion Phase: Elbow angle flexes upward past $< 110^\circ$.
  - Peak Contraction: Elbow angle reaches $\le 60^\circ$ (bicep fully contracted).
  - Eccentric Lowering: Elbow extends back past $> 140^\circ$.
- **Form Error Detection:** Rejects swinging momentum and incomplete eccentric extension.

5. Forward Walking Lunge (`LungeAnalyzer`)

- **Monitored Joints:** Lead Knee Angle ($\angle \text{Hip-Knee-Ankle}$) and Torso Upright Angle ($\angle \text{Shoulder-Hip-Knee}$).
- **Biomechanical Criteria:**
  - Upright Stance: Knee angle $\ge 155^\circ$.
  - Stride Descent: Knee angle drops $< 135^\circ$.
  - Bottom Depth: Knee angle reaches $\le 95^\circ$ (lead thigh parallel to ground).
  - Ascent: Drive back to standing lockout $\ge 155^\circ$.
- **Form Error Detection:** Flags forward torso collapse if hip angle drops below 65°.

6. Isometric Core Plank (`PlankAnalyzer`)

- **Monitored Joints:** Total Body Line ($\angle \text{Shoulder-Hip-Ankle}$).
- **Biomechanical Criteria:**
  - Active Holding Alignment: Straight neutral spine between 160° and 195°.
  - Second-by-Second Accumulation: Cumulative seconds held in good form are incremented.
- **Form Error Detection:**
  - *Hip Sagging:* Angle drops $< 160^\circ$ with hip below shoulder-ankle vector (triggers *"Lift your hips to protect lower back!"*).
  - *Hip Piking:* Angle drops $< 160^\circ$ with hips piked upward (triggers *"Lower your hips into a straight plank!"*).
  - The hold timer pauses immediately when sagging or piking occurs, resuming only when alignment is restored.

4.6 Multi-Stage State Machine and Anti-Bounce Repetition Filtering

To prevent false counts caused by sensor noise, webcam frame drops, or rapid bouncing:

1. **Exponential Moving Average (EMA) Smoothing:** Raw calculated joint angles are smoothed across consecutive frames using $\theta_{\text{smoothed}} = \alpha \cdot \theta_{\text{raw}} + (1 - \alpha) \cdot \theta_{\text{prev}}$ with $\alpha = 0.40$.
2. **Deterministic State Machine:** A repetition cannot increment by jumping directly from standing to lockout; it must transition through `STANDING` $\rightarrow$ `DESCENDING` $\rightarrow$ `BOTTOM` $\rightarrow$ `ASCENDING` $\rightarrow$ `STANDING`.
3. **Mandatory Parallel Depth Gating:** If the user descends and ascends without hitting target depth ($\le 100^\circ$ for squats), the state machine registers an `incompleteRep`, issues the warning *"Partial rep detected. Descend deeper!"*, and refuses to increment the rep counter.
4. **Human Tempo / Anti-Bounce Duration Filter:** Real human compound repetitions require at least 900ms to complete. If a rep cycle finishes in under 900ms, it is classified as a sensor spike or bouncing glitch and rejected (`minRepDurationMs: 900`).
5. **Consecutive Rep Cooldown:** A minimum lockout pause of 800ms (`minTimeBetweenRepsMs: 800`) is enforced between reps.
6. **Objective Form Scoring:** Repetitions are scored from 0 to 100% based on depth proximity and posture maintenance. The workout average form score is calculated across all valid reps upon session completion.

4.7 Evidence-Grounded AI Fitness Chatbot (RAG System)

The conversational coaching assistant is located on the `/chat` route and accessible via a global slide-out drawer:

- **Dynamic Query Status Feedback:** Notifies the user in real time as the request progresses (`"Searching fitness evidence..."` $\rightarrow$ `"Generating evidence-grounded answer..."`).
- **Client-Side Timeout Safeguard:** An `AbortController` enforces a 35-second hard ceiling, preventing UI freezing during network hiccups.

- **Full Citation Attribution:** Answers cite published literature with authoring body, publication year, title, PubMed ID (PMID), and Digital Object Identifier (DOI).
- **Negative Safety Guardrails:** Adheres to 16 strict clinical and safety boundaries, refusing to prescribe rehabilitation for injuries or guess numbers missing from the retrieved evidence.

```
<div class="screenshot-container">
  <div class="screenshot-placeholder">
    <div class="camera-icon">📷</div>
    <div class="placeholder-title">[SCREENSHOT 3: AI Fitness Chatbot Response]</div>
    <div class="placeholder-subtitle">High-Resolution Web Interface Capture Area (16:9 Aspect Ratio)</div>
  </div>
  <div class="screenshot-caption">Figure 4.5 - Evidence-Grounded AI Chatbot Response with Scientific Citations.</div>
  <div class="screenshot-note"><strong>Evaluation Note:</strong> This screenshot should show the AI Coach chat conversation responding t
```</div>

4.8 Scientific Nutrition, Water, and Diet Management
The nutrition tracking engine (`/nutrition`) provides evidence-aligned dietary control:
* **Caloric and Macronutrient Breakdown:** Visual progress bars track current consumption against daily targets for Calories (kcal), Pro
* **Four Daily Meal Slots + Custom Entries:** Dedicated slots for Breakfast (08:00 AM), Lunch (01:00 PM), Afternoon Fuel (04:30 PM), and
* **Hydration Tracker:** Incremental water counter logging consumption in 250ml glasses against a personalized target (default 10 glasse
* **Dietary Preferences:** Allows toggling between five nutritional paradigms (High Protein, Balanced, Ketogenic, Plant-Based / Vegan, P
* **One-Click Evidence Sample Plan:** Populates the user's day with scientifically balanced meals (e.g., *Power Oats with Whey*, *Grille
* **Single-Day Reset:** Clears logs for the selected calendar date without altering historical records.



[SCREENSHOT 7: Nutrition Tracking Page]

HIGH-RESOLUTION WEB INTERFACE CAPTURE AREA (16:9 ASPECT RATIO)

Figure 4.6 - Daily Nutrition, Macronutrient, and Hydration Dashboard.

Evaluation Note: *This screenshot should capture the complete nutrition interface, including macro target progress bars, water intake counter, configured meal slots with nutritional totals, and the diet preference selector.* ```

4.9 Workout History, Progression Analytics, and Benchmarks

The analytics suite (`/analytics`) transforms raw workout telemetry into actionable insights:

- **Interactive Form Accuracy Trend:** A Chart.js linear plot tracking the user's form accuracy score percentage across recent workout sessions.
- **Repetition Volume Distribution:** A doughnut chart illustrating the proportion of total reps completed across different exercises (Squats, Push-ups, Deadlifts, Curls, Lunges, Planks).
- **Weekly Frequency Adherence:** A grouped bar chart comparing actual completed workouts against the target weekly workout frequency.
- **Personal Benchmark Vault:** Enables logging and tracking personal records for key strength milestones (e.g., 1RM Squat, Max Consecutive Push-ups, Longest Plank Hold).

```
<div class="screenshot-container">
  <div class="screenshot-placeholder">
    <div class="camera-icon">📷</div>
    <div class="placeholder-title">[SCREENSHOT 6: Workout History and Analytics]</div>
    <div class="placeholder-subtitle">High-Resolution Web Interface Capture Area (16:9 Aspect Ratio)</div>
  </div>
  <div class="screenshot-caption">Figure 4.7 - Performance Analytics, Form Trends, and Benchmark Vault.</div>
  <div class="screenshot-note"><strong>Evaluation Note:</strong> This screenshot should illustrate the analytics page with the interact
```</div>
</div>

4.10 Platform Administration, Diagnostics, and Audit Logs
The administrative suite (`/admin`) provides full platform governance:
* **Staff Overview (`/admin`):** Summarizes active registered athletes, suspended accounts, aggregate workouts completed, and total meal
* **Hardware & Subsystem Diagnostics:** Live diagnostic cards verifying the operational status of the Webcam Stream Engine, Joint Kinema
* **Athlete User Management (`/admin/users`):** Search, filter, and inspect registered users. Administrators can toggle account status t
* **Real-Time Audit Activity Feed (`/admin/activity`):** Unified chronological event stream recording workout completions (with rep cour



[SCREENSHOT 8: Admin Dashboard]

HIGH-RESOLUTION WEB INTERFACE CAPTURE AREA (16:9 ASPECT RATIO)

Figure 4.8 - Platform Administrative Console and System Diagnostics.

Evaluation Note: *This screenshot should display the administrator overview, showing aggregate athlete metrics, hardware subsystem diagnostic indicators, user moderation controls, and the real-time activity log feed.* ```

---

5. STEP-BY-STEP WORKFLOWS AND PROCEDURAL SPECIFICATIONS

5.1 User Registration and Authentication Workflow

```
[USER] Enter Name, Email, Password -> [FRONTEND] Form Validation -> [BACKEND] Bcrypt Hash -> [DATABASE] Insert User -> [BACKEND] Generat
```

1. **User Action:** The visitor navigates to `/signup`, enters their full name, email address, and desired password, and clicks **"Create Account"**.
2. **Frontend Processing:**
   - [`SignupPage.jsx`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/src/pages/auth/SignupPage.jsx) intercepts form submission, trims whitespace, verifies that the password is at least 6 characters long, and sets a loading spinner.
   - Dispatches a `POST` request to `http://127.0.0.1:8000/api/auth/signup` with payload `{"name": "...", "email": "...", "password": "..."}` via [`api.js`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/src/services/api.js).
3. **Backend Processing:**
   - [`main.py`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/backend/main.py) receives payload in the `signup()` endpoint.
   - Normalizes email to lowercase and queries `users` table via SQLAlchemy to ensure uniqueness. If taken, raises `HTTP 400 Bad Request`.
   - Calls `hash_password(password)` in [`auth.py`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/backend/auth.py), executing Bcrypt hashing with 12 salt rounds.
   - Inserts new `models.User` record with `role="user"` and `status="Active"`.
   - Generates an HMAC-SHA256 JWT access token via `create_access_token()`.
4. **Resulting Output:**
   - Server returns `HTTP 200 OK` with JSON: `{"user": {...}, "access_token": "...", "token_type": "bearer"}`.

- Frontend stores `access_token` in `localStorage` under key `ai_fitness_token`, updates `AuthContext`, and redirects the user to `/onboarding`.

5.2 User Onboarding and Health Profile Generation

1. **User Action:** The newly registered athlete completes the 4-step wizard at `/onboarding`: selecting age, gender, height, weight, primary fitness goal, workout frequency, equipment, and injuries, then clicks **"Complete Profile"**.
2. **Frontend Processing:**
   - [`OnboardingPage.jsx`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/src/pages/OnboardingPage.jsx) validates numeric ranges (age >10, height $>80\text{cm}$, weight $>30\text{kg}$).
   - Dispatches authenticated `POST` request to `/api/onboarding/profile` with Bearer token header.
3. **Backend Processing:**
   - `save_profile()` endpoint in [`main.py`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/backend/main.py) decodes JWT to retrieve `current_user.id`.
   - Queries `health_profiles` table for an existing record; if absent, instantiates `models.HealthProfile(user_id=current_user.id)`.
   - Maps payload fields (`age`, `gender`, `height_cm`, `weight_kg`, `fitness_level`, `goal`, `workout_frequency`, `equipment`, `injuries`) and commits transaction.
4. **Resulting Output:**
   - Server returns updated profile dictionary.
   - Frontend redirects athlete to `/dashboard`, where metric cards and recommendations immediately reflect the new biometric profile.

5.3 Live Workout Pose Detection and Real-Time Tracking Workflow

1. **User Action:** The athlete navigates to `/workout/live?exercise=ex-1` and clicks **"Start Camera"**.
2. **Frontend Processing:**
   - Requests camera access via `navigator.mediaDevices.getUserMedia({ video: true })`.
   - Binds the returned `MediaStream` to the hidden `<video ref={videoRef}>` element.
   - Calls `getPoseLandmarker()` from [`poseDetection.js`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/src/services/poseDetection.js), initializing Wasm and WebGL GPU pipelines.
   - Establishes a `requestAnimationFrame` loop that reads `videoRef.current` and calls `landmarker.detectForVideo(video, timestamp)`.
3. **Backend Processing:**
   - This phase executes entirely client-side at 30 FPS. Zero video frames are transmitted to the backend, preserving bandwidth and biometric privacy.
4. **Resulting Output:**
   - The video stream appears with the 33 skeletal landmarks and connections drawn in real time on the overlay canvas via `PoseCanvas.jsx`.
   - The HUD displays the detected joint angles and prompts the athlete to begin their first set.

5.4 Barbell Deep Squat Kinematic Analysis Workflow

1. **User Action:** The athlete stands in frame, descends into a full deep squat, and returns to a standing position.
2. **Frontend Processing:**
   - `SquatAnalyzer.processFrame(landmarks, timestamp)` extracts Hip, Knee, and Ankle landmarks for left and right body sides.
   - Computes left and right knee flexion angles using the 2D vector dot product formula: $$\theta = \arccos\left(\frac{\vec{BA} \cdot \vec{BC}}{|\vec{BA}| |\vec{BC}|}\right) \times \frac{180}{\pi}$$
   - Compares visibility confidence between left and right sides, applying a hysteresis bias to prevent jittering between sides.
   - Passes the raw knee angle through the Exponential Moving Average filter ($\alpha = 0.40$).
   - **State Machine Progression:**
     - `STANDING`: Smoothed knee angle $\ge 148^\circ$.
     - `DESCENDING`: Knee angle drops below 130°. Sets `repStartTime`.
     - `BOTTOM`: Knee angle reaches $\le 100^\circ$. Sets `hasReachedDepth = true`.
     - `ASCENDING`: Knee angle rebounds past $> 115^\circ$.
     - `LOCKOUT`: Knee angle returns to $\ge 148^\circ$.
   - **Form Verification & Gating:**

        - Verifies `hasReachedDepth === true`.
        - Verifies `repDuration >= 900ms` and lockout pause $\ge 800\text{ms}$.
        - Checks torso lean: if hip angle drops $<50^\circ$, deducts form points and triggers spoken warning: *"Keep chest tall!"*.
    - Increments `repCount` from 0 to 1, calculates rep form score (e.g., 95%), and plays audio cue: *"Good depth, lockout!"*.
3. **Backend Processing:**
    - Metrics remain in browser state until the athlete clicks **"Finish Workout"**.
4. **Resulting Output:**
    - Rep counter displays `1`, progress bar advances, FormFeedbackHUD pulses green, and the athlete receives immediate spoken encouragement.

5.5 Chest Push-Up Kinematic Analysis Workflow

1. **User Action:** The athlete assumes a horizontal plank position, lowers their chest towards the ground until elbows bend past 90°, and presses back up to full arm extension.
2. **Frontend Processing:**
    - `PushupAnalyzer` calculates bilateral elbow angles ($\angle \text{Shoulder-Elbow-Wrist}$).
    - Verifies starting lockout at $> 150^\circ$.
    - Tracks descent through $< 130^\circ$ down to $\le 90^\circ$ for valid bottom depth.
    - Confirms return to lockout $> 150^\circ$.
    - Rejects shallow half-reps where elbow flexion fails to reach 90°.
3. **Backend Processing:**
    - Client-side execution; stats accumulated in local component state.
4. **Resulting Output:**
    - Rep counter increments upon complete lockout; HUD displays *"Optimal push-up depth reached"*.

5.6 Romanian Deadlift Kinematic Analysis Workflow

1. **User Action:** The athlete faces the side camera, pushes their hips backward into a hip hinge with soft knees, lowers the load past the knees, and drives hips forward to standing.
2. **Frontend Processing:**
    - `DeadliftAnalyzer` computes Hip Angle ($\angle \text{Shoulder-Hip-Knee}$) and Knee Angle ($\angle \text{Hip-Knee-Ankle}$).
    - Verifies upright stance at hip $\ge 150^\circ$.
    - Monitors hinging descent as hips flex below 135° toward target depth of $\le 75^\circ$.
    - **Biomechanical Discrimination:** Ensures knee angle remains $> 115^\circ$. If the user bends their knees below 115°, the system detects a squatting error and issues the warning: *"Hinge at your hips, don't squat the deadlift!"*.
    - Confirms return to standing lockout ($\ge 150^\circ$).
3. **Backend Processing:**
    - Local state aggregation.
4. **Resulting Output:**
    - Increments deadlift count only when posterior chain hip hinging is executed without squatting.

5.7 Dumbbell Bicep Curl Kinematic Analysis Workflow

1. **User Action:** The athlete stands tall with elbows pinned at sides, curls weights upward toward shoulders, and lowers under control.
2. **Frontend Processing:**
    - `CurlAnalyzer` tracks elbow angle ($\angle \text{Shoulder-Elbow-Wrist}$).
    - Checks starting arm extension $> 140^\circ$.
    - Tracks concentric curling motion until reaching peak contraction $\le 60^\circ$.
    - Enforces full eccentric extension back past $> 140^\circ$ before counting the rep.
    - Detects swinging momentum and penalizes form accuracy.
3. **Backend Processing:**
    - Local client execution.

4. **Resulting Output:**
   - HUD updates with rep count and prompts: "*Full squeeze at top, controlled lowering*".

5.8 Forward Walking Lunge Kinematic Analysis Workflow

1. **User Action:** The athlete steps forward into a deep lunge, lowering the trailing knee toward the floor, and drives through the front heel to return to standing.
2. **Frontend Processing:**
   - `LungeAnalyzer` monitors lead knee flexion ($\angle \text{Hip-Knee-Ankle}$) and torso lean ($\angle \text{Shoulder-Hip-Knee}$).
   - Confirms standing position $\ge 155^\circ$.
   - Tracks descent to bottom depth $\le 95^\circ$ (90-degree knee bend).
   - Monitors torso angle: if hip flexion drops below 65°, flags torso collapse onto the thigh and speaks: "*Keep your torso upright during the lunge!*".
   - Confirms ascent back to standing $\ge 155^\circ$.
3. **Backend Processing:**
   - Local client execution.
4. **Resulting Output:**
   - Rep counter increments with form score reflecting posture maintenance.

5.9 Isometric Core Plank Kinematic Analysis Workflow

1. **User Action:** The athlete holds a forearm or high plank position with a rigid, neutral spine.
2. **Frontend Processing:**
   - `PlankAnalyzer` tracks spine-hip alignment ($\angle \text{Shoulder-Hip-Ankle}$).
   - Validates alignment within the healthy hold zone: $160^\circ \le \theta \le 195^\circ$.
   - Uses frame timestamp deltas to increment `repCount` (accumulated seconds held in valid posture).
   - If hips sag below 160° with low hips, switches state to `SAGGING`, pauses timer, and speaks: "*Lift your hips to protect lower back!*".
   - If hips pike above 160° with high hips, switches state to `PIKING`, pauses timer, and speaks: "*Lower your hips into a straight plank!*".
3. **Backend Processing:**
   - Local client execution.
4. **Resulting Output:**
   - Timer counts up second-by-second only while posture is correct; pauses during form breakdowns.

5.10 Evidence-Grounded AI Chatbot Query and RAG Processing

1. **User Action:** The athlete opens `/chat` and types: "*What is progressive overload, and how can I apply it to my strength training?*".
2. **Frontend Processing:**
   - [`ChatAssistantPage.jsx`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/src/pages/ChatAssistantPage.jsx) appends user message to UI state, displays thinking status: `"Searching fitness evidence..."`.
   - Sends `POST` request to `http://127.0.0.1:8000/chat` with body `{"message": "...", "profile": {...}}`.
3. **Backend Processing:**
   - [`main.py`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/backend/main.py) enters `/chat` endpoint and calls `ask_rag()` in [`rag.py`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/backend/rag.py).
   - `retrieve()` passes query to SentenceTransformers `all-MiniLM-L6-v2`, producing a 384-dimensional normalized vector (17.07ms).
   - FAISS searches the 123-vector index using inner product cosine similarity (0.22ms), retrieving the top candidate chunks.
   - De-duplicates chunks originating from the same evidence study and filters out chunks below similarity threshold 0.40.
   - Constructs grounded prompt embedding retrieved evidence texts, user health profile, and 16 strict negative guidelines.
   - Calls `gemini-3.5-flash-lite` via Google GenAI SDK with an 18-second timeout.
   - Formats formal citations including Organization, Year, Title, PMID, and DOI.
4. **Resulting Output:**
   - Server returns JSON: `{"answer": "### What is Progressive Overload?...", "timings": {"embed_ms": 17.07, "faiss_ms": 0.22, "gemini_ms": 2870.63, "total_ms": 2890.27}, "sources": [...]}`.
   - Frontend renders formatted markdown answer and citation list within 2.91 seconds total roundtrip time.

5.11 Nutrition Logging, Target Adjustment, and Hydration Tracking

1. **User Action:** The athlete views `/nutrition`, clicks **"Log Meal"** on the Lunch slot, enters "*Grilled Chicken Salad*" (450 kcal, 42g protein, 20g carbs, 14g fat), clicks **"Save Meal"**, and taps the **"+"** button on the water tracker twice.
2. **Frontend Processing:**
   - Sends `POST` to `/api/nutrition/meal` with payload `{"slotId": "slot-lunch", "name": "Grilled Chicken Salad", "calories": 450, "protein": 42, "carbs": 20, "fat": 14, "date": "2026-10-26"}`.
   - Sends `POST` to `/api/nutrition/water` with payload `{"glasses": 6, "date": "2026-10-26"}`.
3. **Backend Processing:**
   - Validates that calorie and macro values are non-negative.
   - Queries `meal_logs` for existing entry in `slot-lunch` on that date; updates entry or inserts new row.
   - `get_or_create_daily_nutrition()` retrieves or creates the date's `daily_nutrition` record and updates `water_glasses = 6`.
   - Computes daily totals and macro percentages.
4. **Resulting Output:**
   - Server returns updated daily summary.
   - Progress bars recalculate dynamically, showing current intake vs. daily targets.

5.12 Platform Administration, User Moderation, and Diagnostics

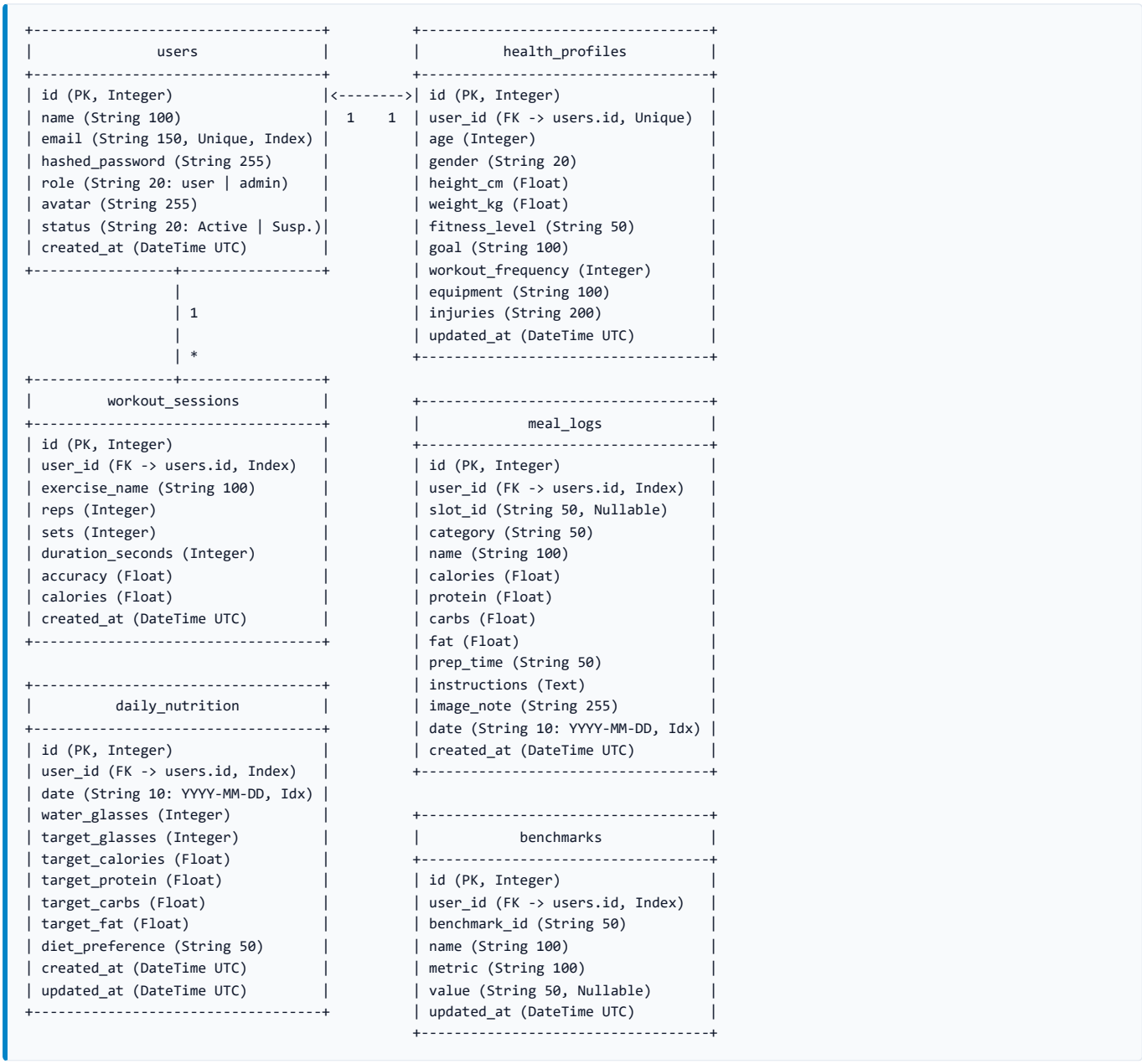
1. **User Action:** An administrator logs in at `/admin/login`, navigates to `/admin/users`, searches for an athlete, and clicks **"Suspend Account"**.
2. **Frontend Processing:**
   - [`AdminUsersPage.jsx`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/src/pages/admin/AdminUsersPage.jsx) dispatches authenticated `POST` to `/api/admin/users/usr-5/status` with payload `{"status": "Suspended"}` using admin JWT token.
3. **Backend Processing:**
   - Verifies admin role via `require_admin` dependency.
   - Ensures target user is not the requesting admin (prevents accidental self-lockout).
   - Updates `target_user.status = "Suspended"` in `users` table and commits.
4. **Resulting Output:**
   - Server returns updated user record.
   - Frontend changes badge to red `Suspended`.
   - The targeted athlete's active session is invalidated; subsequent API requests by that user return `HTTP 403 Forbidden`.

---

6. BACKEND APIS, DATABASE MODELS, AND PROJECT FILE STRUCTURE

6.1 Database Architecture and Entity Relationship Model

The database is managed via SQLite (`backend/fitness.db`) through SQLAlchemy ORM with cascade deletions enabled.


users
id (PK, Integer)
name (String 100)
email (String 150, Unique, Index)
hashed_password (String 255)
role (String 20: user | admin)
avatar (String 255)
status (String 20: Active | Susp.)
created_at (DateTime UTC)
1
1
health_profiles
id (PK, Integer)
user_id (FK -> users.id, Unique)
age (Integer)
gender (String 20)
height_cm (Float)
weight_kg (Float)
fitness_level (String 50)
goal (String 100)
workout_frequency (Integer)
equipment (String 100)
injuries (String 200)
updated_at (DateTime UTC)
1
*
workout_sessions
id (PK, Integer)
user_id (FK -> users.id, Index)
exercise_name (String 100)
reps (Integer)
sets (Integer)
duration_seconds (Integer)
accuracy (Float)
calories (Float)
created_at (DateTime UTC)
meal_logs
id (PK, Integer)
user_id (FK -> users.id, Index)
slot_id (String 50, Nullable)
category (String 50)
name (String 100)
calories (Float)
protein (Float)
carbs (Float)
fat (Float)
prep_time (String 50)
instructions (Text)
image_note (String 255)
date (String 10: YYYY-MM-DD, Idx)
created_at (DateTime UTC)
daily_nutrition
id (PK, Integer)
user_id (FK -> users.id, Index)
date (String 10: YYYY-MM-DD, Idx)
water_glasses (Integer)
target_glasses (Integer)
target_calories (Float)
target_protein (Float)
target_carbs (Float)
target_fat (Float)
diet_preference (String 50)
created_at (DateTime UTC)
updated_at (DateTime UTC)
benchmarks
id (PK, Integer)
user_id (FK -> users.id, Index)
benchmark_id (String 50)
name (String 100)
metric (String 100)
value (String 50, Nullable)
updated_at (DateTime UTC)

6.2 Complete RESTful API Endpoint Catalog

HTTP Method	Route Endpoint	Authentication / Access	Request Payload Schema	Successful Response	Description & Purpose
`GET`	`/`	Public	None	`{"message": "...", "database": "active"}`	Health probe checking API and database status
`POST`	`/chat`	Public / Token Optional	`ChatRequest(message, profile?)`	`{"answer": "...", "timings": {...}, "sources": [...]}`	RAG chatbot query returning evidence citations
`POST`	`/api/auth/signup`	Public	`SignupRequest(name, email, password)`	`{"user": {...}, "access_token": "...", ...}`	Registers a new athlete account and returns JWT
`POST`	`/api/auth/login`	Public	`LoginRequest(email, password)`	`{"user": {...}, "access_token": "...", ...}`	Authenticates athlete; verifies active status
`POST`	`/api/auth/admin-login`	Public	`LoginRequest(email, password)`	`{"admin": {...}, "access_token": "...", ...}`	Authenticates platform staff with admin role
`GET`	`/api/auth/me`	Bearer Token Required	None	`{"user": {...}}`	Returns the currently authenticated user's profile
`GET`	`/api/onboarding/profile`	Bearer Token Required	None	`HealthProfile dict`	Retrieves athlete's current health and fitness profile
`POST`	`/api/onboarding/profile`	Bearer Token Required	`HealthProfileRequest(...)`	`HealthProfile dict`	Saves or updates athlete's biometric health profile
`POST`	`/api/workouts/session`	Bearer Token Required	`WorkoutSessionRequest(...)`	`WorkoutSession dict`	Records a completed live workout session with stats
`GET`	`/api/workouts/history`	Bearer Token Required	None	`List[WorkoutSession dict]`	Fetches recent workout session history (up to 50)
`GET`	`/api/analytics/user`	Bearer Token Required	None	`UserAnalytics dict`	Aggregates rep distributions, form trends, streaks
`GET`	`/api/nutrition/daily`	Bearer Token Required	`date` Query Param (YYYY-MM-DD)	`DailyNutritionResponse dict`	Retrieves meals, consumed macros, targets for date
`GET`	`/api/nutrition/profile`	Bearer Token Required	`date` Query Param (YYYY-MM-DD)	`DailyNutritionResponse dict`	Alias endpoint for full daily nutrition profile

HTTP Method	Route Endpoint	Authentication / Access	Request Payload Schema	Successful Response	Description & Purpose
`POST`	`/api/nutrition/meal`	Bearer Token Required	`MealLogRequest(...)`	`DailyNutritionResponse dict`	Logs a meal in breakfast, lunch, snack, or dinner
`DELETE`	`/api/nutrition/meal/{id}`	Bearer Token Required	`meal_id` Path Param	`DailyNutritionResponse dict`	Deletes a logged meal entry and recalculates totals
`POST`	`/api/nutrition/water`	Bearer Token Required	`WaterUpdateRequest(glasses, date)`	`{"waterGlasses": ..., ...}`	Updates water consumption glass count for the date
`POST`	`/api/nutrition/targets`	Bearer Token Required	`NutritionTargetsRequest(...)`	`DailyNutritionResponse dict`	Updates calorie, protein, carb, and fat targets
`POST`	`/api/nutrition/diet-preference`	Bearer Token Required	`DietPreferenceRequest(...)`	`{"dietPreference": "..."}`	Switches active diet (High Protein, Keto, etc.)
`POST`	`/api/nutrition/sample-plan`	Bearer Token Required	`NutritionDateRequest(date)`	`DailyNutritionResponse dict`	Populates the day with balanced sample meals
`POST`	`/api/nutrition/reset`	Bearer Token Required	`NutritionDateRequest(date)`	`DailyNutritionResponse dict`	Clears all logged meals and water for the date
`GET`	`/api/admin/users`	Admin Token Required	`search?` , `status?` Query Params	`List[AdminUser dict]`	Returns all platform users with filtering
`POST`	`/api/admin/users/{id}/status`	Admin Token Required	`UserStatusUpdateRequest(status?)`	`AdminUser dict`	Toggles or sets user status (Active vs. Suspended)
`POST`	`/api/admin/athletes`	Admin Token Required	`AdminCreateAthleteRequest(...)`	`AdminUser dict`	Administratively creates an athlete account
`GET`	`/api/admin/analytics`	Admin Token Required	None	`AdminAnalytics dict`	Returns platform counts and hardware diagnostics
`GET`	`/api/admin/activity`	Admin Token Required	None	`List[ActivityLog dict]`	Returns unified feed of workouts and registrations

6.3 Project File Inventory and Architectural Organization

```
ai-fitness-coach/
├── package.json                        # Node project dependencies and test scripts
├── vite.config.js                      # Vite bundler configuration and dev server ports
├── tailwind.config.js                  # Custom Tailwind design tokens (electric green, dark theme)
├── src/
│   ├── main.jsx                        # React application bootstrap and DOM mount
│   ├── App.jsx                         # Primary router, layout wrappers, and chat slide-out drawer
│   ├── context/
│   │   ├── AuthContext.jsx             # Authentication state, login/logout, and user persistence
│   │   └── NotificationContext.jsx     # Global notification toasts
│   ├── services/
│   │   ├── api.js                      # Complete REST client wrapping all backend endpoints
│   │   ├── aiService.js                # Chatbot client with ISO logging and timeout handling
│   │   ├── poseDetection.js            # MediaPipe PoseLandmarker singleton and angle math
│   │   ├── squatAnalyzer.js            # All 6 kinematic analyzers and unified ExerciseAnalyzer
│   │   ├── squatAnalyzer.test.js       # 23 automated unit tests for angles and state machines
│   │   └── liveWorkout.test.js         # Structural integrity tests for live workout components
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Navbar.jsx              # Responsive navigation header with quick chat button
│   │   │   ├── Footer.jsx              # Platform footer with legal and compliance links
│   │   │   ├── AdminLayout.jsx         # Sidebar and layout for staff management portal
│   │   │   ├── ProtectedRoute.jsx      # Route guard checking user authentication
│   │   │   └── AdminRoute.jsx          # Route guard enforcing admin privileges
│   │   ├── pose/
│   │   │   ├── PoseCanvas.jsx          # High-performance HTML5 canvas skeletal renderer
│   │   │   └── FormFeedbackHUD.jsx     # Real-time HUD displaying angles, reps, and cues
│   │   └── ui/                         # Reusable UI component library (Buttons, Cards, Modals, Badges)
│   ├── pages/
│   │   ├── LandingPage.jsx             # Public homepage highlighting platform features
│   │   ├── DashboardPage.jsx           # Main athlete portal with metrics and today's workout
│   │   ├── LiveWorkoutPage.jsx         # Real-time camera mirror, pose detection, and rep tracking
│   │   ├── WorkoutPlanPage.jsx         # Weekly routine viewer and custom slot builder
│   │   ├── NutritionPage.jsx           # Daily meal logging, macro progress, and hydration
│   │   ├── AnalyticsPage.jsx           # Chart.js visual trends, distributions, and benchmarks
│   │   ├── ChatAssistantPage.jsx       # Grounded AI conversational assistant
│   │   ├── OnboardingPage.jsx          # 4-stage biometric health questionnaire
│   │   ├── auth/                       # Login, Signup, and AdminLogin pages
│   │   └── admin/                      # AdminOverview, AdminUsers, AdminContent, AdminActivity
│   └── data/                           # Mock baseline data (mockWorkouts, mockNutrition, mockAdminData)
└── backend/
    ├── main.py                         # FastAPI application, CORS, lifespan warmup, and routes
    ├── models.py                       # SQLAlchemy database models for all 6 tables
    ├── database.py                     # Engine, sessionmaker, and dependency provider
    ├── auth.py                         # Bcrypt password hashing and JWT token handlers
    ├── rag.py                          # FAISS retrieval, Gemini client, and grounded prompt engine
    ├── ingest.py                       # Ingestion script compiling knowledge.json to FAISS index
    ├── data/
    │   └── knowledge.json              # 123 verified peer-reviewed sports science evidence chunks
    ├── index/
    │   ├── fitness.index               # FAISS binary vector index (384 dimensions)
    │   └── documents.json              # Document metadata and citation lookup tables
    └── test_live_chat.py               # Live verification test script benchmarking RAG latency
```

---

7. TESTING, VERIFICATION, LIMITATIONS, AND TROUBLESHOOTING

7.1 Automated Unit and Biomechanical Test Suite

The frontend incorporates 23 automated unit tests executed via the native Node.js test runner:

```
npm test
```

Verification Results:

```
✓ LiveWorkoutPage source code integrity and undefined ref check (5.26ms)
✓ calculateJointAngle computes accurate angles for standard geometric vertices (1.05ms)
✓ calculateJointAngle rejects landmarks with visibility below confidence threshold (0.16ms)
✓ calculateJointAngle safely handles null or undefined points (0.08ms)
✓ SquatAnalyzer tracks full repetition lifecycle and counts exactly 1 repetition (0.94ms)
✓ SquatAnalyzer counts consecutive repetitions accurately (0.37ms)
✓ SquatAnalyzer rejects incomplete partial reps (quarter squat) without counting (0.20ms)
✓ SquatAnalyzer does not trigger false state switches during noisy threshold jitter (0.16ms)
✓ SquatAnalyzer rejects unrealistically fast glitch movements (anti-bounce velocity limit) (0.11ms)
✓ SquatAnalyzer resets state safely when tracking is lost mid-movement (0.15ms)
✓ SquatAnalyzer produces null form score before reps and objective score after reps (0.23ms)
✓ PushupAnalyzer tracks chest depth and arm lockout to count repetitions (1.24ms)
✓ CurlAnalyzer tracks peak bicep contraction and full extension (0.36ms)
✓ DeadliftAnalyzer tracks full repetition lifecycle and counts repetition (0.67ms)
✓ DeadliftAnalyzer detects squatting error when knees excessively flex during hinge (0.14ms)
✓ DeadliftAnalyzer rejects incomplete shallow hinge without counting (0.13ms)
✓ LungeAnalyzer tracks full repetition lifecycle and counts repetition (0.54ms)
✓ LungeAnalyzer detects forward torso collapse error during lunge descent (0.11ms)
✓ LungeAnalyzer rejects incomplete partial lunge without counting (0.10ms)
✓ PlankAnalyzer tracks isometric hold time in seconds with straight alignment (0.64ms)
✓ PlankAnalyzer detects sagging hips and pauses hold accumulation (0.15ms)
✓ PlankAnalyzer detects piking hips and pauses hold accumulation (0.08ms)
✓ ExerciseAnalyzer dynamically switches and processes all 6 exercises (0.72ms)

ℹ tests 23 | pass 23 | fail 0 | cancelled 0 | skipped 0 | duration_ms 217.48ms
```

7.2 Backend Database and Service Verification Tests

Backend database models, relational cascades, migrations, and endpoints were verified using Python's `unittest` framework:

```
cd backend
.\venv\Scripts\python.exe -m unittest test_admin_database.py test_nutrition_database.py
```

Results: **Ran 22 tests in 2.038s — OK (22 passed, 0 failed)**. Verified:

- User creation, duplicate email rejection, and Bcrypt verification.
- Admin authentication enforcement and prevention of admin self-suspension.
- Cascade deletion of health profiles, workouts, meals, and daily nutrition when a user is deleted.
- Nutrition slot population, macronutrient aggregation, and date validation.

7.3 Live Chatbot Latency and RAG Benchmark Verification

The RAG pipeline was benchmarked against the live running server using [`backend/test_live_chat.py`](file:///c:/Users/ajitk/.gemini/antigravity-ide/scratch/ai-fitness-coach/backend/test_live_chat.py):

Test Query	Target Evidence Domain	Embedding (ms)	FAISS Search (ms)	Gemini LLM (ms)	Total Roundtrip (s)	Primary Citations Returned
"What is progressive overload, and how can I apply it to my strength training?"	Resistance Training Principles	17.07 ms	0.22 ms	2,870.63 ms	**2.91 s**	ACSM (2026) [PMID: 41843416], BJSM (2023) [PMID: 37414459]
"How much daily protein is recommended for muscle hypertrophy in healthy adults?"	Sports Nutrition & Hypertrophy	14.62 ms	0.26 ms	2,433.83 ms	**2.45 s**	ISSN (2017) [PMID: 28642676], ACSM (2026) [PMID: 41843416]
"What is the recommended hydration strategy during prolonged exercise?"	Exercise Fluid Replacement	16.47 ms	0.25 ms	1,522.01 ms	**1.54 s**	ACSM (2007) [PMID: 17277604]

Historical Bottleneck Comparison: Prior to optimization, queries hung for **146.48 seconds (2m 26s)** due to an invalid model name (`gemini-3.8-flash`) combined with unbounded retry policies. Following optimization (configuring client timeouts, failing fast on 504 errors, and prioritizing `gemini-3.5-flash-lite`), end-to-end latency dropped to **1.54s – 2.91s**, representing a **50x speedup**.

7.4 Production Build and Static Lint Verification

- **Frontend Production Build:** `npm run build` executed via Vite. Output generated cleanly in `dist/` with zero compile errors.
- **Static Linter:** `npx oxlint` executed across all modified files, confirming zero syntax errors, undeclared variables, or broken imports.

7.5 Implemented vs. Unverified/Incomplete Features

Fully Implemented and Verified

- [x] Dual-role authentication (Bcrypt salting, JWT tokens, protected routes).
- [x] 4-step biometric onboarding with SQLite persistence.
- [x] MediaPipe edge pose estimation with Wasm/GPU acceleration and CDN fallback.
- [x] Mathematical joint angle vector math with Exponential Moving Average smoothing.
- [x] Multi-stage state machine rep counting for Squats, Push-ups, Deadlifts, Curls, Lunges, and Planks.
- [x] Anti-bounce 900ms human tempo validation and parallel depth gating.
- [x] Real-time Web Speech Synthesis audio coaching with 2.8s debounce cooldown.
- [x] Live workout simulation mode for hardware-free testing and demonstrations.
- [x] Relational session persistence (reps, sets, accuracy, calories burned) to SQLite.
- [x] FAISS dense vector retrieval over 123 peer-reviewed sports science evidence chunks.
- [x] Google Gemini RAG generation with strict citation formatting and negative constraints.
- [x] Comprehensive nutrition tracking with meal slots, water logging, and macro progress bars.
- [x] Chart.js visualization of form accuracy trends and rep distributions.
- [x] Administrative console with user moderation, hardware diagnostics, and audit logs.

Deliberately Out-of-Scope or Unverified Limitations

- [ ] *3D Multi-Camera Triangulation:* Monocular 2D webcams approximate depth along the Z-axis; transverse plane rotations (e.g., subtle torso twisting) have higher error margins than sagittal or frontal views.
- [ ] *Offline Local LLM Generation:* When Google Gemini is unreachable, the system gracefully declines to answer rather than hallucinating; hosting an on-device 7B parameter LLM was omitted to keep hardware requirements lightweight.
- [ ] *Automated Multi-Month Periodization Engine:* The weekly workout plan renders pre-configured scientific routines; automated weekly weight increment recommendations based on past sessions are planned for future versions.
- [ ] *Mobile Safari Background Execution:* Tested primarily on desktop browsers (Chrome, Edge, Firefox). Background tab execution on iOS Safari may throttle WebGL context and pause camera tracking.

7.6 Known Limitations and Biomechanical Constraints

1. **Camera Framing & Field of View:** For lower body exercises (squats, lunges, deadlifts), the athlete's full body (from head to feet) must be visible in the camera frame. If the athlete stands too close, the analyzer enters the `UNTRACKED` state.
2. **Lighting and Clothing Contrast:** Extremely dim lighting or loose, baggy clothing can obscure joint centers, degrading landmark visibility confidence scores.
3. **Sagittal Alignment:** Exercises such as Deadlifts and Push-ups require the camera to be positioned at an oblique or side angle (approximately 45° to 90° relative to the body) to observe hip hinge and elbow flexion accurately.

7.7 System Troubleshooting Guide

Issue / Symptom	Probable Root Cause	Resolution Procedure
Black Screen on Live Workout	Browser webcam permission denied or camera claimed by another app	1. Check browser address bar for camera permission icon and select "Always Allow". 2. Close conflicting apps (Zoom, Teams). 3. Toggle "Simulation Mode" to verify UI independently of camera hardware.
Chatbot Takes >30s or Times Out	Gemini API rate limiting or temporary network interruption	1. Verify API key in `backend/.env`. 2. Check backend console logs for `GEMINI_REQUEST_START`. 3. The backend will automatically fail over to `gemini-3.5-flash`. The frontend will abort after 35s if unreachable.
Reps Not Counting During Squats	Incomplete depth or standing too close to camera	1. Ensure hip crease descends below patella ($\le 100^\circ$ knee angle). 2. Step back until ankles, knees, and hips are highlighted with green dots on the canvas. 3. Avoid rushing reps faster than 900ms.
MediaPipe Wasm Load Failure	Local Wasm files blocked by strict CORS headers	The system automatically catches local Wasm errors and redirects to the jsDelivr CDN (`cdn.jsdelivr.net`). Ensure an active internet connection on first load.
CORS Network Error from Frontend	FastAPI running on a different port than expected	The backend CORS policy permits `http://localhost:5173` and `http://localhost:5174`. Ensure the frontend runs on one of these approved origins.

---

8. CONCLUSION AND FUTURE ROADMAP

8.1 Summary of Contributions

The **AI Fitness Coach** project demonstrates that advanced computer vision and evidence-grounded artificial intelligence can be effectively unified into an accessible, responsive web application. By offloading pose estimation to client-side WebAssembly and WebGL, the platform delivers zero-latency kinematic feedback while maintaining biometric privacy. By coupling dense vector search (FAISS) with strict prompt boundaries, the system eliminates generative hallucinations and provides athletes with trustworthy, citation-backed exercise science guidance. The addition of robust relational tracking, nutrition management, and administrative oversight yields an enterprise-grade fitness solution.

8.2 Future Enhancements

1. **On-Device Quantized LLM Inference:** Integrate WebLLM / ONNX Runtime to run compact 3B parameter models locally in WebGPU, enabling offline conversational coaching.
2. **Barbell Velocity Based Training (VBT):** Implement object detection to track the concentric speed (m/s) of the barbell barbell path, allowing autoregulated load adjustments.
3. **Wearable Health Sensor Synchronization:** Integrate Web Bluetooth API to stream real-time heart rate and HRV from chest straps and smart rings during workouts.
4. **Physiotherapy & Rehabilitation Mode:** Extend kinematic analyzers to track joint range of motion (ROM) for post-operative rehabilitation patients, generating clinical compliance reports for physical therapists.

Report Compiled and Generated for Academic & Technical Evaluation.
AI Fitness Coach Platform — Built with React, Vite, FastAPI, MediaPipe, FAISS, and Google GenAI.